

How the database works

A practical guide to the data flow, workflows, SQL automation, and web app

Database: pet_adoption_network
Server: local MySQL 8.0
Web app: Flask on localhost:5000 when started

Prepared from the project source files and the database loaded into MySQL 8.0.

1. System overview

Layer	Role
Browser	A person opens a Flask page and submits a form or selects a view.
Flask application	Routes handle the request, run parameterized SQL, and render HTML templates.
MySQL Connector	Python opens a MySQL connection using settings from config.py and environment variables.
MySQL database	Tables store records; constraints, views, triggers, and procedures enforce or simplify database work.

Typical read path: Browser request -> Flask route -> SQL query -> MySQL result rows -> HTML template -> browser response. For a write, Flask commits the transaction after inserting/updating data.

2. Main workflows

1 Register the animal

The animals page lists animals and allows a new record to be added. The shelter link is optional. The database validates required fields, status values, and the shelter reference.

2 Find a match

An adopter's preferred species and size are compared with animals whose current status is Available. Matching results can show the shelter and basic care information.

3 Track the application

An application connects one adopter with one animal. Its status can be Pending, Under Review, Approved, Rejected, or Withdrawn. Home visits add observations and a recommendation.

4 Place in foster care

A fostering row links an animal to a foster home and tracks dates and status. A trigger checks active placements against the home's maximum capacity.

5 Complete an adoption

The procedure checks the application, most recent home visit, recommendation, and animal availability before recording a completed adoption in a transaction.

6 Follow medical care

Medical records store examination and next check-up details. The follow-up view labels pending care as overdue or upcoming. Alerts can be generated when records are inserted or when the alert procedure is run.

3. How the SQL features behave

Tables and constraints

Primary keys uniquely identify rows. Foreign keys connect related rows, such as each medical record to an animal. Checks and enums reject invalid values. Indexes support common lookups by status, species/size, application status, or follow-up date.

Views

Views are saved SELECT statements. **Available_Animals_View** joins available animals with shelter details. **Medical_Followup_View** calculates a follow-up state from the check-up date and follow-up status. **Adoption_Application_Summary_View** combines application, animal, adopter, and home-visit information.

Triggers

- A newly inserted pending medical record with a past next-check-up date can create an alert.
- A completed adoption record updates the animal's current status to Adopted.
- Creating or reactivating an active foster placement checks the foster home's capacity; completion makes the animal Available again.

A trigger runs because a database row changes. It does not wake up just because a date passes. Run **Generate_Overdue_Medical_Alerts()** to check older records for newly overdue follow-ups.

Stored procedures and transactions

Approve_Adoption(application_id) begins a transaction, locks the application row, checks that the application is approved, requires a completed and recommended home visit, and rejects an already-adopted animal. If checks pass, it writes the adoption record and commits. On a failed prerequisite, it rolls back and returns an error.

Animal_Availability_Report() returns the available-animal view. **Generate_Overdue_Medical_Alerts()** inserts missing alerts and returns a count.

4. What you can do in the web app

Page	Purpose
Dashboard	Counts of animals by status, shelters, pending applications, and overdue follow-up.
Animals	Browse animals or add a new animal record.
Shelters / Adopters	Review shelter and adopter records.
Medical Follow-up	Review next check-up dates and follow-up states.
Applications	Review application status alongside visit outcome and recommendation.
Adoptions	Review completed adoption records.
Matching	Choose an adopter and display currently available animals matching species/size preferences.

5. How to start the application

The SQL database has already been imported locally. The Flask application is separate and requires Python dependencies. From the extracted project folder, use the README setup sequence:

Step	Command / action
1	Create and activate a Python virtual environment: <code>py -m venv .venv</code> , then <code>.venv\Scripts\activate</code>
2	Install dependencies: <code>pip install -r requirements.txt</code>
3	Copy <code>.env.example</code> to <code>.env</code> and set <code>DB_PASSWORD</code> for the local MySQL account. Keep this file private.
4	Start the app: <code>python app.py</code>
5	Open <code>http://127.0.0.1:5000</code> in a browser.

The application defaults to host 127.0.0.1, port 3306, user root, and database `pet_adoption_network`. These can be changed with `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, and `DB_NAME` environment settings.

6. Direct SQL examples

Run these in MySQL Workbench after selecting the `pet_adoption_network` schema:

Task	Example SQL
List available animals	<code>SELECT * FROM Available_Animals_View;</code>
See application details	<code>SELECT * FROM Adoption_Application_Summary_View;</code>
See follow-up state	<code>SELECT * FROM Medical_Followup_View;</code>
List animal counts by species	<code>SELECT species, COUNT(*) FROM animals GROUP BY species;</code>
Generate overdue medical alerts	<code>CALL Generate_Overdue_Medical_Alerts();</code>
List matching available animals	<code>CALL Animal_Availability_Report();</code>

Adoption procedure usage: `CALL Approve_Adoption(application_id);` The procedure intentionally rejects applications that fail its prerequisites.

7. Troubleshooting

Symptom	Check
App cannot connect	Confirm MySQL80 is running, database name is correct, and .env credentials match the local account.
No matching animals	Confirm the animal is Available and its species/size matches the adopter preferences.
Adoption procedure errors	Check Approved status, a Completed + Recommended home visit, and that the animal has no completed adoption.
Overdue alert is missing	Confirm follow-up is Pending and the next check-up date is in the past; run the alert generation procedure.
Port 5000 is unavailable	Stop another app using the port or configure Flask to use another local port.

Data safety: the supplied sample data is suitable for learning and demonstration. Use backups and access controls before changing real operational records.

Prepared from the project's SQL scripts, Flask routes, and README. The project uses parameterized SQL for values submitted by the web application.